KU LEUVEN

FACULTY OF SCIENCE

Cycling Patterns in Flanders

Modern Data Analytics

Prof. Martial Luyts, Ruben Kerkhofs, Grégory Van Kruijsdijk

Fausto De Valck (r0892545), Tobias Vanoverschelde, Alexander Lievens, Timon Knol

Academic year **2025–2026**

Contents

1 Introduction

Cycling is a cornerstone of sustainable mobility in Flanders. To plan and maintain its cycling infrastructure, the *Agentschap Wegen en Verkeer* (AWV) operates a network of automatic counting stations across the region. These stations record cyclist volumes at 15-minute intervals, producing a rich publicly available dataset that opens the door to a quantitative understanding of cycling behaviour.

This project moves beyond a descriptive view of *when* and *where* people cycle, and asks what *drives* cycling volume. We combine the AWV bicycle counts with daily weather observations from the Open-Meteo API, Belgian public holidays, and calendar features, and we fit a tree-based regression model whose feature importances and partial dependences serve as the analytical backbone of our explanations. Results are presented through an interactive Shiny for Python dashboard so that stakeholders can explore the patterns themselves.

We formulate three research questions, taken directly from the project proposal:

RQ1. What drives cycling volume in Flanders? We quantify the relative contribution of weather, temporal and location features to daily cycling counts.

RQ2. How do cycling patterns vary across time and space? We identify peak hours, weekly and seasonal trends, and spatial variation across municipalities and counting sites.

RQ3. How sensitive is cycling to weather conditions? We estimate how much a one-degree change in temperature, or a one-millimetre change in precipitation, affects the predicted daily count.

The remainder of this report is structured as follows. Section 2 describes the data and our cleaning steps. Section 3 summarises the exploratory analysis. Section 4 details the methodology: feature engineering, modelling pipeline, and interpretation techniques. Section 5 presents the results and answers the three research questions. Section 6 introduces the dashboard, Section 7 covers the infrastructure and deployment choices, and Section 8 concludes with limitations and directions for future work.

2 Data

2.1 AWV bicycle counts

The primary data source consists of automatic bicycle counts collected by EcoCounter sensors across Flanders, publicly available through the Flemish open data portal. The data are published in three linked CSV files:

- **Counts** (`data-YYYY-MM.csv`): one monthly file per month from August 2019 to April 2026, with 15-minute interval observations recording the number of cyclists, pedestrians, or other road users detected at each station. Each row contains the site identifier, direction (IN, OUT, or IN/OUT), road user type, start and end timestamps, and count value.
- **Sites** (`sites.csv`): metadata for each counting station, including its WGS84 latitude and longitude, name, municipality (*gemeente*), road number, district, and installation date.
- **Directions** (`richtingen.csv`): metadata describing the directions recorded at each station.

For this project, we filter strictly to cyclist counts (`type = FIETSERS`), discarding pedestrian, automobile and other categories.

2.2 External enrichment

Cycling is highly sensitive to weather, so we enrich the dataset with daily weather observations from the Open-Meteo historical weather API[1] for the full date range of the cycling data. The variables retained are temperature at 2 m above ground, precipitation, wind speed at 10 m, and cloud cover. Because querying weather per individual site would inflate the API load by approximately 75×, we use a single representative weather series for Flanders; the spatial variation in weather across the region is small relative to the variation in cycling counts.

We further engineer two categorical context variables: **COVID periods**, defined manually as five non-overlapping intervals (pre-COVID, first lockdown, summer 2020, second lockdown, post-COVID), and **Belgian public holidays**, derived programmatically using the `holidays` Python package.

2.3 Cleaning and aggregation

After aggregation from the raw 15-minute records to hourly resolution per site and direction, the cleaned dataset contains approximately 10.4 million observations. We attach the site metadata and detect outliers using `IsolationForest` (`contamination=0.01`), which flags 1.00 % of the rows (103 704 in absolute terms). These rows are dropped from the analysis. For modelling, we further aggregate to a daily level per (date, site, gemeente) tuple, summing the cyclist counts and averaging the weather variables. The resulting analytical base table contains **217 192 daily rows**, covering the period August 2019 through April 2026.

3 Exploratory analysis

3.1 Temporal patterns

Hourly profiles reveal a clear distinction between weekdays and weekends. Weekdays show two sharp commuting peaks, one around 08:00 and one around 17:00, consistent with the work and school cycle. Weekends, by contrast, show a single, broader afternoon peak, reflecting leisure cycling.

At the weekly scale, counts on Tuesdays through Thursdays are typically the highest, with Mondays and Fridays slightly lower, and weekends substantially lower at most counting stations. At the seasonal scale, cycling volume follows a clear sinusoidal pattern with a peak in late spring and summer and a trough in winter.

3.2 COVID-19 impact

The five COVID periods leave a visible imprint on the data. The first lockdown (mid-March to early May 2020) suppressed counts at commuter-heavy sites while increasing them at leisure-oriented locations. The summer of 2020 saw a rebound, after which the second lockdown depressed counts again. The post-COVID period sees a recovery in commuter cycling, though not always to pre-pandemic levels at all sites.

3.3 Spatial variation

Counting stations in city centres (notably Leuven, Gent, and Brussels-periphery municipalities) consistently record the highest daily volumes, while rural sites record substantially lower counts. The municipality (*gemeente*) and the precise location of the sensor (`lat`, `lon`) therefore both carry strong predictive information.

[1] https://open-meteo.com

3.4 Weather correlations

Temperature correlates positively with cycling counts: warmer days bring more cyclists. Precipitation has a strong negative effect, with even moderate rainfall (5–10 mm) measurably reducing counts. Wind speed and cloud cover have smaller but consistent negative associations.

These exploratory observations motivate the feature set used in the modelling stage and provide initial intuition against which the model's feature importances and partial dependences can be compared.

4 Methodology

4.1 Feature engineering

The model receives the following feature groups, all engineered through reusable functions in the `src/features.py` module:

- **Cyclical temporal encodings.** Day-of-week and month are encoded as paired sin / cos features, ensuring that Sunday and Monday (or December and January) are correctly represented as adjacent rather than maximally distant.
- **Calendar flags.** Boolean flags for weekend, morning rush (07:00–09:00) and evening rush (16:00–18:00), plus raw `hour`, `day_of_week`, `month`, and `year` integers.
- **COVID period.** A five-level categorical encoding of the pandemic timeline.
- **Belgian holiday flag.** A boolean indicating whether the date is a recognised public holiday.
- **Spatial features.** The WGS84 latitude and longitude of each counting site, joined from `sites.csv`. These allow the model to learn geographic proximity directly, complementing the one-hot encoded municipality.
- **Weather features.** Daily temperature, precipitation, wind speed, and cloud cover from Open-Meteo.

4.2 Modelling pipeline

We use a scikit-learn `Pipeline` composed of two stages. The first is a `ColumnTransformer` that applies `StandardScaler` to the numeric and cyclical features, `OneHotEncoder(handle_unknown="ignore")` to the categorical features (`gemeente`, `covid_period`), and passthrough for boolean features. The second stage is a regression estimator. By wrapping everything in a single pipeline, we ensure that the same preprocessing is applied identically at training and prediction time, eliminating a common source of leakage.

We split the data temporally: training on observations before the 80th percentile of dates (cutoff: **2025-06-28**) and testing on the remainder. This yields 173 627 training rows and 43 565 test rows. Inside the training set we use `GridSearchCV` with `TimeSeriesSplit` (five folds) so that hyperparameter selection respects the temporal ordering. Models are selected on the basis of mean absolute error (MAE) on the cross-validation folds.

We compare three regression families:

- **Ridge regression**, as a linear baseline.
- **Random Forest**, as a non-linear baseline.
- **HistGradientBoosting**, a fast histogram-based variant of gradient-boosted trees that handles missing values natively and scales well on tabular data of this size.

All stochastic estimators are seeded with `random_state = 42` so a re-run reproduces the same model selection and the same test metrics.

4.3 Interpretation techniques

In line with the proposal, the model's role is *explanatory* rather than purely predictive. Two complementary interpretation techniques are used.

Permutation importance. The classical tree-based "feature importance" attribute (the sum of split gains attributable to a feature) is well known to be biased towards features with high cardinality, in our case the one-hot encoded `gemeente`, which provides the model with many split opportunities. We therefore use `sklearn.inspection.permutation_importance`, which shuffles each feature in turn on a held-out sample and measures the drop in R^2. The resulting ranking is unbiased with respect to cardinality and is comparable across feature types.

Partial dependence plots (PDP). To answer Research Question 3, we compute partial dependences for `temperature_2m`, `precipitation`, and `wind_speed_10m` using `sklearn.inspection.partial_dependence`. A PDP varies one feature across its observed range while averaging over the empirical distribution of the others, giving the marginal effect of that feature on the prediction. The slope of the temperature PDP, for example, directly answers the proposal question "how much does a degree of temperature change affect cycling counts".

5 Results

5.1 Model comparison

Table 1 summarises the held-out test performance of the four candidate models. HistGradientBoosting outperforms the others on all three metrics.

Table 1: Test-set performance of the three models.

Model	Test MAE	RMSE	R^2
Ridge	124.3	185.7	0.77
Random Forest	168.9	228.9	0.65
HistGradientBoosting	**94.3**	**154.2**	**0.84**

The best HistGradientBoosting configuration uses `learning_rate = 0.1`, `max_iter = 500` and `max_depth = 7`, selected by five-fold `TimeSeriesSplit` cross-validation on training MAE.

These numbers reflect training on the dataset *after* the IsolationForest outlier filter is applied. Removing those rows improves both MAE and RMSE across all three models. The R^2 values are slightly lower than in earlier iterations of this pipeline; this is the expected effect of dropping the upper tail of the target distribution, which reduces the total variance against which the residual variance is compared.

A predicted-vs-actual scatter plot of the test set (see Appendix, Figure A1) shows a tight diagonal cluster for moderate counts (≤ 1000 cyclists per day) and increased scatter at higher counts, consistent with the model under-predicting the most extreme summer peaks. $R^2 = 0.89$ is high for a noisy domain like cycling, where idiosyncratic events (cycling festivals, infrastructure works, exam periods) drive unmodelled variation.

5.2 What drives cycling volume? (RQ 1)

The permutation importance ranking (Figure A2) places the **categorical municipality** far ahead of every other feature: `gemeente` captures persistent local cycling cultures that no temporal or weather feature can replicate. The next two slots are **latitude** and **longitude**, which together describe the geographic position of the counting station. A second tier of similarly important features follows:

temperature and the cyclic day-of-week encoding (`day_of_week_sin`), with **precipitation** just behind. The weekly cycle is the strongest purely temporal driver, and within the weather block temperature matters more than rain.

The latitude and longitude features absorb part of the variance that would otherwise be carried by the one-hot `gemeente` columns; two nearby municipalities are recognised as similar by the model, where the one-hot encoding treats them as fully independent.

The COVID-period categorical contributes essentially nothing once the other features are present, with a permutation importance indistinguishable from zero. This does not mean COVID had no effect on cycling (the exploratory analysis shows that it clearly did), but the within-period variation is already well explained by weather, calendar and location, so the period label adds no further signal.

5.3 How do cycling patterns vary across time and space? (RQ 2)

Spatially, the model assigns substantially higher predicted counts to city-centre stations in Leuven, Gent and Brussels-adjacent municipalities than to rural ones. The addition of `lat/lon` allows the model to interpolate to areas with few observations, capturing the gradient from urban to rural rather than treating each gemeente as an isolated category. Temporally, the day-of-week and month cyclic encodings, combined with the weekend and holiday flags, reproduce the two-peak weekday pattern, the broader weekend pattern, and the strong seasonal cycle observed in Section 3.

5.4 How sensitive is cycling to weather? (RQ 3)

The partial dependence plots (Figure A3) yield direct quantitative answers. Over the observed range of roughly 1–21 °C the temperature PDP is approximately linear with an average slope of about +10 cyclists per day per degree, for a total swing of around +190 cyclists between the coldest and warmest days in the test set.

Precipitation has the expected negative effect, but the shape is not linear. The first few millimetres of rain account for most of the drop, after which the PDP plateaus; the total swing across the observed range is around −80 cyclists per day.

Wind speed and cloud cover have smaller effects: roughly −2 cyclists per day per additional metre per second of wind, and about −0.6 cyclists per day per additional percent of cloud cover. Over the full observed range that adds up to about −35 and −55 cyclists respectively. These magnitudes are consistent with the low permutation importances of the two features.

6 Dashboard

To make these findings accessible we built an interactive dashboard in Shiny for Python (`app.py`), organised into three tabs.

Tab 1: Temporal explorer. The user selects one or more years and a municipality (or "All Flanders") and is shown the hourly cycling profile, split by weekday and weekend, together with day-of-week and monthly aggregate views. This tab directly supports Research Question 2.

Tab 2: Model insights. The user sees the top N feature importances of the trained model and a predicted-vs-actual scatter plot on the test set. This tab supports Research Question 1 by exposing the same ranking that backs Section 5.

Tab 3: Weather impact. The user sets sliders for temperature, precipitation, wind speed, weekday and municipality, and immediately sees the model's predicted daily count for the scenario. Three plots accompany the prediction: a temperature sweep holding the other inputs fixed, the population-level partial-dependence curve, and an observed-data scatter for the chosen municipality. This tab supports Research Question 3.

The dashboard reads the pre-computed `feature_importance.parquet` and `pdp_results.parquet` artifacts produced by `src/training.py`, rather than recomputing them on each request, keeping the user interface responsive.

7 Infrastructure and deployment

The project is organised around three reproducibility-oriented design decisions.

First, the data preparation, model training, and interpretation steps are encapsulated in two executable modules (`src/pipeline.py` and `src/training.py`) that together replace the exploratory notebooks for reproducible runs. A single `python -m src.pipeline && python -m src.training` invocation rebuilds every artifact required by the dashboard.

Second, all model runs are tracked with **MLflow**: parameters, cross-validation MAE, test metrics, and the serialised model itself are logged under the `cycling-flanders` experiment. The best model is promoted to the MLflow model registry, from which the Shiny app loads it at startup.

Third, the entire stack is packaged in a multi-stage **Dockerfile**. Stage one builds the Python environment from `requirements-prod.txt` using `uv` for fast dependency resolution. Stage two runs the pipeline and training scripts inside the image to materialise the parquet artifacts and the trained model. Stage three is a slim runtime image that copies only the processed data and the Shiny application, and serves the dashboard on port 8000. The clean separation between training-time and serving-time concerns means rebuilds for UI changes do not retrain the model.

A Google Cloud Vertex AI deployment of the model is under development; the deployment script (`src/deploy_vertex.py`) and an endpoint test harness are in place, and a target deployment to a Raspberry Pi for low-cost continuous hosting is also being explored.

8 Conclusion

We have built an end-to-end analytical pipeline for cycling in Flanders. A HistGradientBoosting regressor trained on six years of AWV counts, enriched with weather, calendar, COVID-period and spatial features, reaches $R^2 = 0.89$ on a held-out temporal test set. Permutation importance identifies municipality, temperature, weekend status and precipitation as the dominant drivers; partial dependence plots quantify the marginal effect of each weather variable; and an interactive Shiny dashboard exposes all of this to a non-technical audience.

Limitations. Daily aggregation throws away two things: the intraday commuter-peak structure that is clearly visible in the exploratory analysis, and the ability to react to short-lived weather events. Open-Meteo already returns hourly data, so an hourly model could answer "does the thunderstorm at 3pm reduce cycling that afternoon" rather than only "does a rainy day reduce cycling that day". We chose daily resolution for narrative simplicity rather than for compute reasons. HistGradientBoosting handles the 10 million hourly rows comfortably, and the cloud-native pipeline can absorb a heavier training run by swapping the VM up a tier.

The weather series is also regional rather than per-site. We pull a single hourly series for a central Flanders point and join it to every site, which gives a roughly 75× reduction in Open-Meteo calls compared to one query per site. Flanders is small enough that the spatial weather variation on a given day is modest relative to the day-to-day variation across the dataset, but the choice still introduces error on days with locally concentrated weather, for example a thunderstorm over Antwerp that the central point does not see.

Outlier flagging is statistical rather than event-based, so genuinely unusual but explicable cycling events (festivals, races, exam weeks) are treated identically to sensor anomalies.

Future work. Hourly modelling, integration of event calendars, and continuous online learning from the monthly AWV updates would all extend the model. On the infrastructure side, completing

the Vertex AI deployment would enable real-time prediction from the dashboard rather than batch prediction from a baked-in model.

A Appendix

Repository. All code is available at
https://github.com/TobiasVanoverschelde/Modern-Data-Analytics---Group-2.

Project structure.

```
src/
  loaders.py          CSV readers for counts, sites, directions
  cleaning.py         filter to cyclists, hourly aggregation, outliers
  weather.py          Open-Meteo API integration
  features.py         cyclical encoding, COVID, holidays, lat/lon
  modeling.py         ColumnTransformer + pipeline + temporal CV
  interpretation.py   permutation importance + partial dependence
  tracking.py         MLflow setup
  pipeline.py         end-to-end data pipeline (replaces nb 01/02)
  training.py         end-to-end training + interpretation (replaces nb 03)
  deploy_vertex.py    Google Cloud Vertex AI deployment
notebooks/            original exploratory notebooks (kept for reference)
app.py                Shiny for Python dashboard
Dockerfile            multi-stage build (deps -> training -> serving)
tests/                pytest smoke tests for features and modelling
```

Figures.

- **Figure A1**: Predicted vs actual scatter plot, HistGradientBoosting on test set (`figures/predicted_vs_actual.`
- **Figure A2**: Top permutation feature importances (`figures/permutation_importance.png`).
- **Figure A3**: Partial dependence of predicted count on temperature, precipitation, and wind speed (`figures/pdp_weather.png`).
- **Figure A4**: Dashboard screenshots (Temporal explorer, Model insights, Weather impact tabs) in `extra/`.

Reproducing the analysis. From a clone of the repository:

```
docker build -t mda-cycling .
docker run --rm -p 8000:8000 mda-cycling
```

Then open http://localhost:8000 in a browser. The build stage downloads the raw data, runs `src/pipeline.py` and `src/training.py`, and bakes the resulting model and parquet artifacts into the serving image.